Trail Information Logging System

The Trailblazers

Nicholas Cabrera	Conlan Connor	Fabi Marrufo
Ethan Patterson	Richard Tang	Trevor Wiesen

Interns

Lucas French, Giovanni Paulino

Adirondack Wilderness Advocates

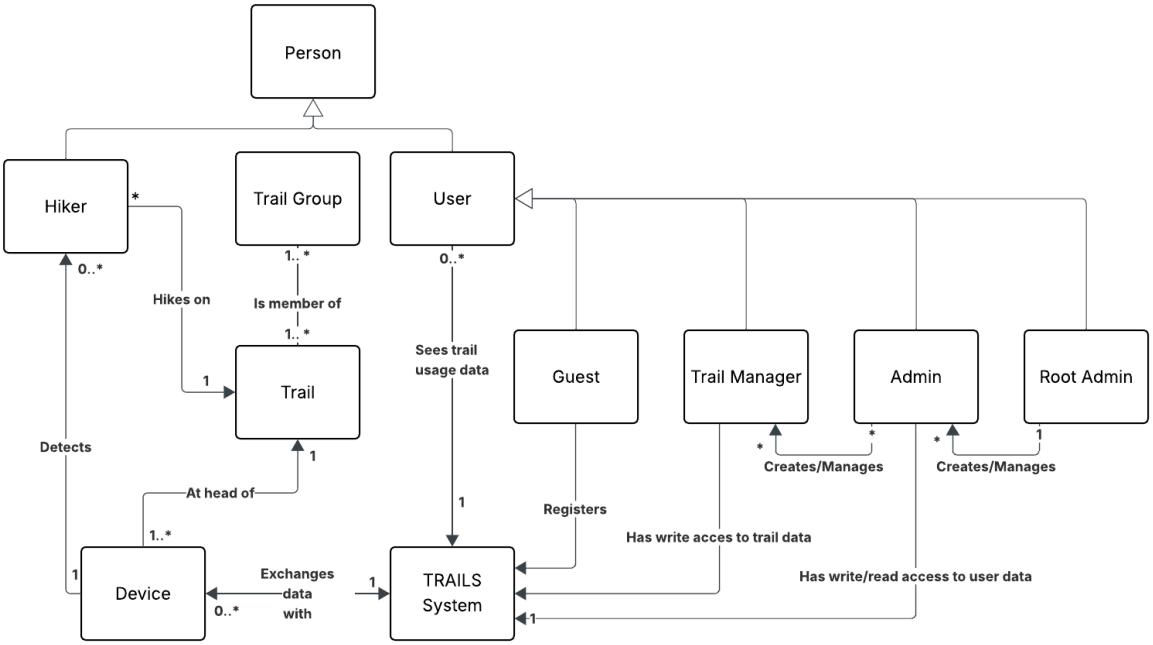
Craig McGowan

Faculty Coach

Dr. Sandeep Mitra

Project Overview

The Adirondack Park is the largest publicly protected area in the contiguous United States, larger than the Grand Canyon, Yosemite, Glacier, and Yellowstone National Parks combined. It is roughly the size of Vermont, and with over 900 trailheads, 2000 miles of trails, and 6 million acres of land the scale of the Adirondack Park, and of this project, cannot be underestimated. The Trail Information Logging System, or TRAILS, is a cloud hosted web application meant to serve various user groups in visualizing aggregated trail usage data that is individually uploaded by devices across the park. Each device is placed at a trailhead, and counts how many hikers pass by with an IR sensor. Devices may or may not be connected to a cellular network, if so they upload their data to the cloud once a day around midnight, otherwise it must be imported manually.



Above is a domain model of the entire system, from the problem domain. Trail Groups are “Areas”, and you will find that most of the project will revolve around the administrators, as they are the primary audience for the sponsor.

Project Requirements

At the very start of the project, during requirements gathering, it was critical to us to establish an extremely clear MVP. This way the team would always know what they were working towards and could plan a sprint or two in advance. This was absolutely essential as many features had to be completely redesigned to take the scale of the project into account. Those that have taken Software Testing will know that defects are easiest to detect and fix at the very first stage of the SDLC, thus we put a lot of effort into static and textual analysis of the requirements, and created a “source of truth” **document**[1] which holds the clarified and defect-less requirements. This document serves also to bridge the gap between what the team plans to build, and what the sponsor expects. It holds wireframes for each page in the application, what features are on the page, what features are restricted to certain privilege groups, and what the user should see depending on what actions they take given their privilege. It also holds client-facing requirements for the domain, mobile optimization, and device emulator, two of which are SCOOP intern projects and will be explained in detail below. It is recommended that when you start this project, that you create a fresh tab in this document and add to it. It will make your life significantly easier, as we used it also as guidelines during implementation. If a developer is confused on what should be done, they should be able to have all their questions answered by this document.

Constraints

At the start of the project, the team inherited a data model developed by a previous team. Because the data model is a foundational component of the system, modifying it posed a significant risk in terms of time and effort. Any changes would require updates across multiple parts of the application, making it a costly decision.

Another key consideration was the use of DynamoDB, a non-relational database chosen by the previous team. As we adapted the data model to meet new requirements, we recognized that a relational database might better support our needs. However, due to time constraints and the complexity of migrating existing data and infrastructure, we decided not to switch. This remains a potential limitation that should be revisited as the system continues to scale.

The project also involved coordination between multiple teams, specifically our software team and a hardware team responsible for developing the trail counter devices. Misaligned schedules, our team working in the spring and summer, while the hardware team operated in the spring and fall, created challenges in communication and synchronization. To mitigate this, we developed a device emulator to simulate hardware interactions. Additionally, changes to our data model required updates to device requirements, further emphasizing the need for stronger cross-team communication, which was not always consistent.

Security was another constraint, as previous teams had not prioritized it. Our team introduced mutual TLS (mTLS), which required multiple development sprints and presented a learning curve for team members with limited cybersecurity experience. We also identified risks such as

exposing AWS keys in the repository, which required remediation to align with security best practices.

Stakeholder involvement introduced additional complexity. The project sponsor was highly engaged and frequently updated requirements, which at times led to scope creep. Furthermore, because the sponsor contributed directly to the shared repository, the team had to carefully manage changes to avoid conflicts and ensure system stability.

Finally, the addition of two interns introduced both opportunities and challenges. While they contributed to development efforts, their limited experience required team members to spend time mentoring and supporting them, which reduced the time available for core development tasks.

Development and Management Process

The team used the AGILE Scrum Methodology, which was requested by the sponsor and the team agreed as it is the methodology we are most familiar with. We decided that a strong process from the very beginning that we kept to consistently was something we wanted to take seriously. Every one of us treated the project with professionalism, and it massively boosted our productivity and effectiveness. There were several different development and management processes. In terms of management, the Project Manager was in charge of running the project, documentation, meeting agendas, coordinating the teams effort during implementation, spearheading the requirements gathering process, and anything else that wasn't expressly the responsibility of the other team members. Running the project entails ensuring that leadership is strong and maintains a clear vision of the MVP at all times to minimize thrashing, that the team does its due diligence and gets deliverables done on time, and that engineering decisions are thought through with the non-functional requirements being taken into account. Documentation entails the actual planning of the project, any RIT deliverables that needed to be written, and any documents that were deemed necessary. If a task had the word "document" in it, that generally fell under the PMs purview. Meeting agendas were heavily structured, starting with a 4-Up that the PM presented to the team to give the team an overall picture of the project's current state, and then was followed up by each member of the team given 1-2 minutes to bring to the team's attention anything they thought people should know about. This usually included pointed status updates, things they wanted to discuss during the meeting, any issues they were having, and so on. After this, the meeting would commence and usually included looking at PRs, reviewing tasks or designs, and doing any implementation that had to be done together. Afterwards, the PM would go through the next steps for the future, and what he wanted to see done by the next meeting. The PM was also the scrum master and in charge of running sprint planning, demonstrations, and assigning team members to tasks. The requirements gathering process was run by the project manager but heavily included the rest of the team, since each requirement usually was some team member's specialty. The requirements gathering meeting usually consisted of going through the project proposal, performing static/textual analysis on the requirement, identifying gaps and defects, and clarifying with the sponsor what they meant. Then, the team created a work breakdown structure to identify what they thought they could reasonably complete with the team they had, and presented a vision for the MVP to the sponsor. This involved some negotiation with the sponsor that the PM was in charge of, and ensured that sponsor expectations were aligned with what the team believed they could reasonably accomplish. The result of this meeting was the Requirements, Flow, and Use Cases document

which fully described what the team believed they should make, how it worked, what it should look like, and was presented to the sponsor to ensure that his expectations were satisfied. Whenever a new requirement was added or changed, then that change needed to be reflected on the document. This was formalized with the introduction of the **Scope and Requirements Change**[3] document. However, it was abandoned by the sponsor in favor of a more agile approach. The team still used the mechanism internally to document large changes to critical requirements, such as data model additions or anything that required an addition to the Requirements, Flow, and Use Cases document.

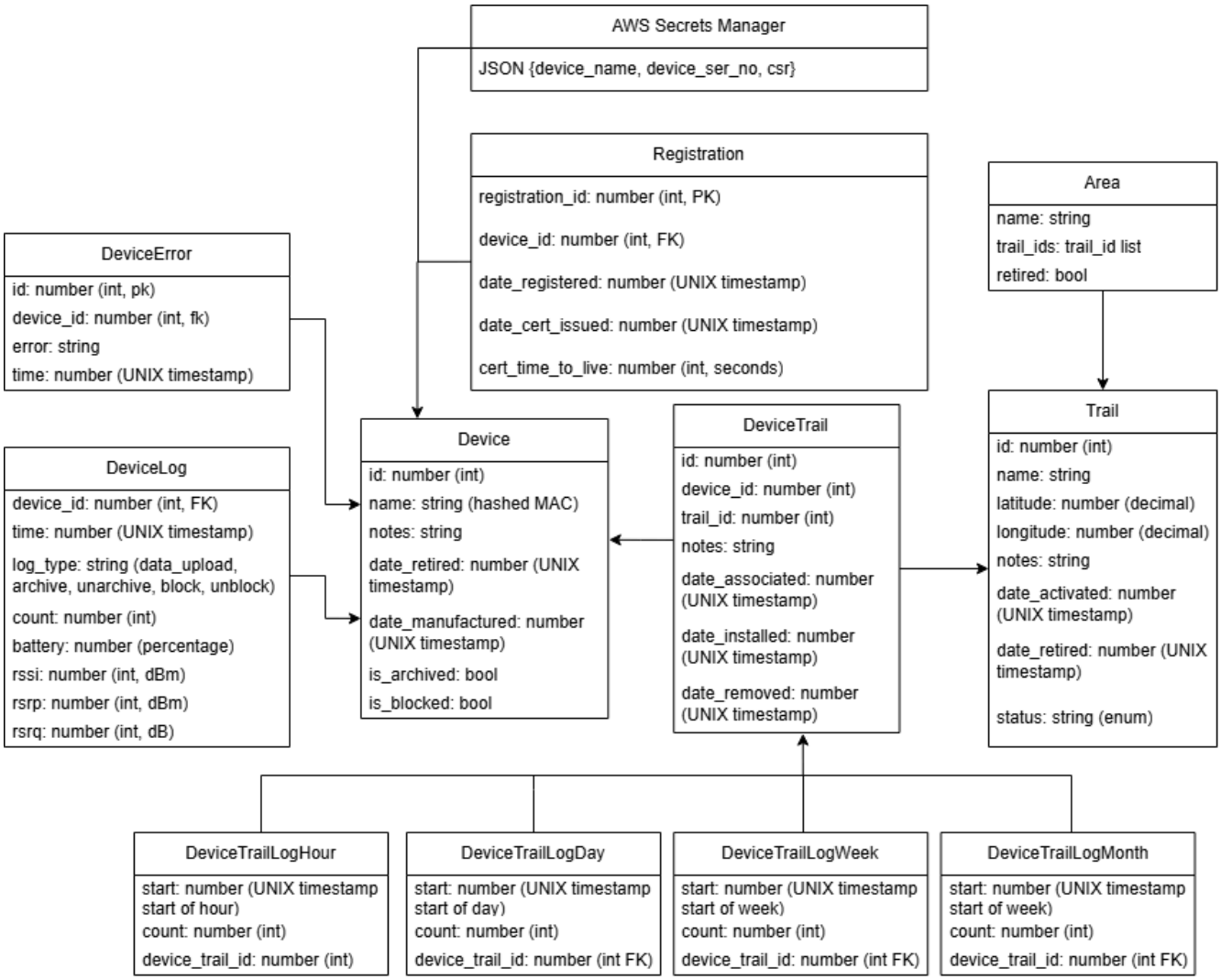
In terms of development process, a similarly strong process was followed. The team had two week long sprints, during which time a task was assigned to a single or multiple team members according to story points and interest from the team member. Originally, there were strict subteams and tasks would be assigned to subteams, however, team members actually found themselves wanting to branch out more and take tasks not necessarily within the scope of their subteam, so the idea was abandoned and tasks were assigned more fluidly based on who wanted to do them. The structure of a sprint was that on Wednesday, the previous sprint's changes would be demoed, and the next sprint would be planned. Tasks were taken off the backlog based on the roadmap and sponsor feedback, planning poker would assign points, and the team and PM would assign tasks. Then, two weeks later on the Monday before the demo day, the team would meet and merge their changes into the TST environment, which helped to see if individual feature branches could be merged with nothing breaking. Any issues were fixed then pushed up to the UAT branch, which was specifically for the sponsor's use to see if what the team had created was in line with expectations. Then the sponsor had from Monday-Wednesday to submit feedback on these changes, and on Wednesday the team would demo the changes to the sponsor to explain the design rationale and intended usage, and to discuss the sponsors feedback. Then the process started over.

Project Schedule: Planned and Actual

The project schedule was heavily influenced by two sources: the MVP defined in the Requirements, Flow, and Use Cases document as well as the **work breakdown structures**[6] created at the beginning of the project. These were then put into a Github Projects and organized into a **roadmap**[7] of all the tasks the team planned to complete and in what order they had to be completed. This way the team could envision everything they had to do for weeks and months in advance, and the sponsor knew at all times the status of the project. This was further refined later on to include sponsor-defined “priorities” that were assigned to tasks that would inform which tasks the team committed to during sprint planning. It turned out that the team completed tasks far faster than was anticipated, and this allowed the team to commit to a few enhancements it did not originally believe it could do. The team was able to complete all MVP requirements plus a heat map view. With the addition of two interns, a mobile optimization enhancement was also completed, as well as a device emulator to further test the device integration.

System Design

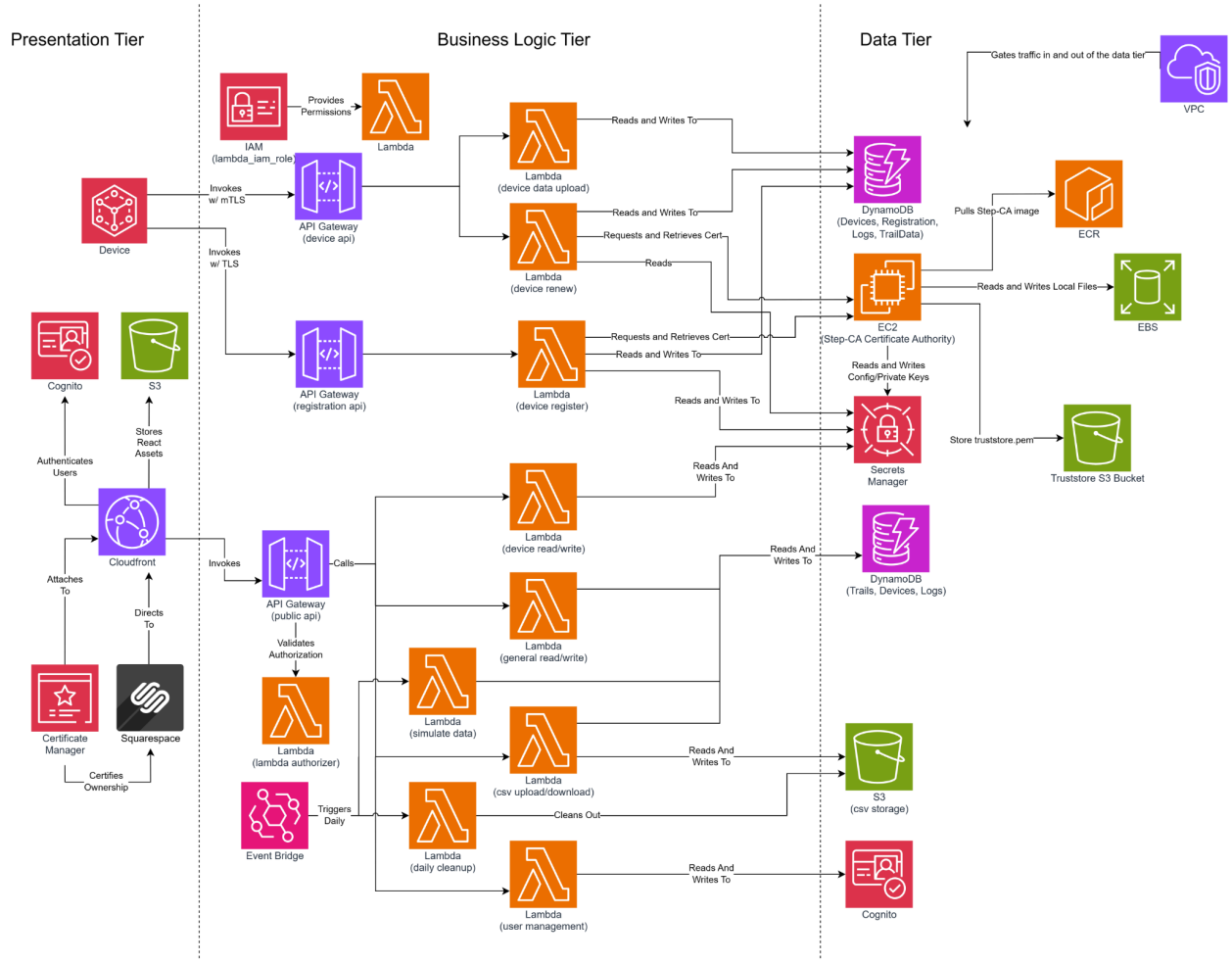
Data Model



The data model is the heart of the web application. It defines the tables that are within the persistence layer (DynamoDB) and are set up non-relationally. This is a departure from what is normally taught in our degree, however this has been very carefully thought out and designed. It is also frozen, only additions and enhancements are allowed as deletion and changes could cause catastrophic errors. The most important part of the table is the DeviceTrail and associated Log at different granularity levels. Notice that this data is redundant, in a relational setup this would be ripe for normalization. However, non-relational databases need to be set up to reduce query time as much as possible, and redundancy is the trade off to do that. We have four different tables for DeviceTrailLogs, one for each granularity available on the web application. This ensures that when a specific granularity is selected on the dashboard, that the database table associated with that granularity is queried and responds as fast as possible. This is done because storing at the lowest granularity and constructing it from there is simply infeasible, we did performance tests given different set ups with different database types, and not only is non-relational the fastest, it is also the cheapest which is an important non-functional requirement. This project is run by a

non-profit organization and paid for out of the sponsors pocket, it's very important that the cost be minimized as much as possible.

Amazon Web Services Design & Architecture



The project's AWS architecture can be broken down into three distinct sections defined by a three-tier microservices architecture, these being the presentation tier, business logic tier, and the data tier. When the team inherited the project, most of the microservices architecture decisions were already made, so the team worked to build off the existing structure instead of creating it from scratch. The original project implemented the architecture using exclusively AWS resources, while the new architecture is similar but with a few modifications and additions.

The presentation tier is where users interact with the application. This remains largely unchanged, using cloudfront to host the application, and Simple Storage Service (S3) which stores the react files that cloudfront references when displaying the application to a user. Users can authenticate with Cognito, and retrieve a token that is sent in alongside any API requests. The only change to the presentation tier is a shift to use squarespace to manage the domain instead of Route53. Route53 is an AWS resource for managing custom domains, where you can buy a domain in Route53, or manage a domain's configuration if it exists outside of AWS. Since

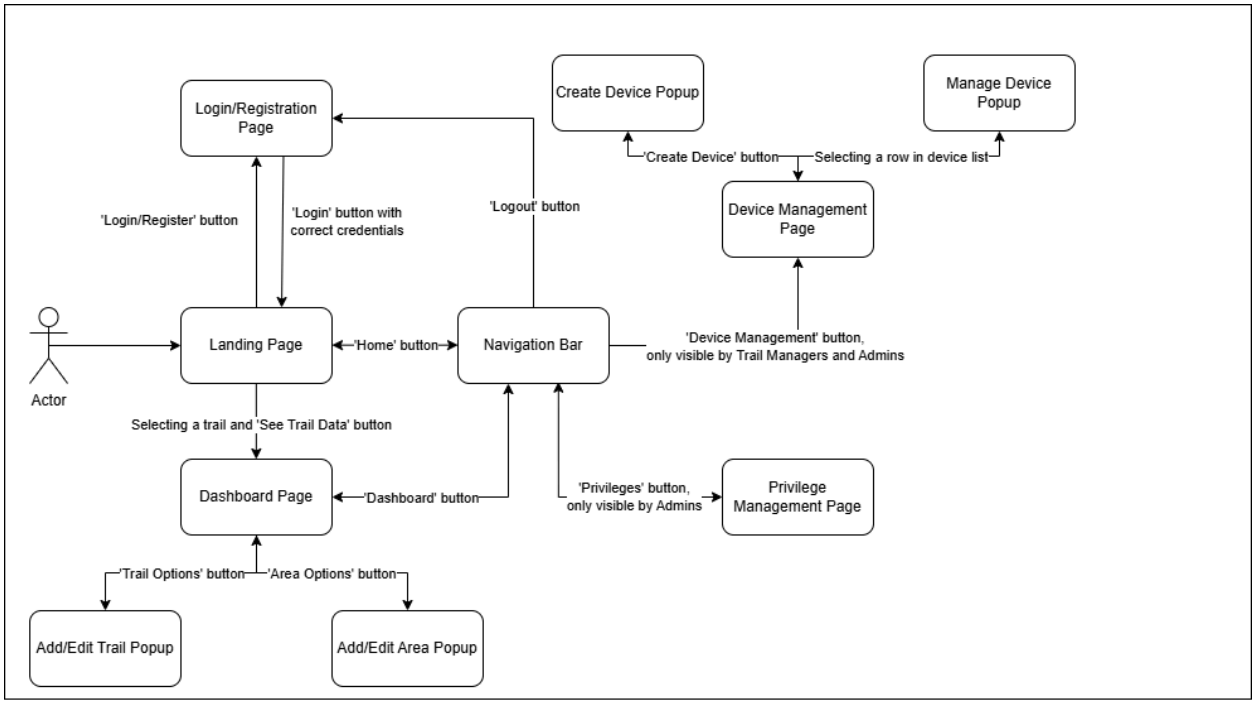
the domain provided by the sponsor exists on Squarespace, and managing DNS records in Squarespace is free with the ownership of a domain, we switched to using Squarespace to manage the DNS records to make management cheaper and easier. Certificate Manager is what AWS uses to validate ownership of our specific domain, allowing us to use the custom domain with Cloudfront, API Gateway, and Simple Email Service (SES).

The business logic tier is where all the business logic resides, translating user actions into application functionality. This is the tier where most changes to the architecture were made. The core of an API Gateway calling Lambda functions remains the same, but the specific API Gateway and Lambdas underwent many changes. The major change within the API Gateway was splitting up the API depending on authentication. Previously devices and users would call the same API, but use different authentication. Since the team implemented mTLS on the devices API calls but not the users, these needed to be separated into different APIs. The Public API, used by the frontend, changed authentication from Cognito token authentication, to be a custom Lambda authorizer. This allowed for different users having access to different endpoints, rather than all users having access to every endpoint.

The Lambdas within the business logic tier also underwent a major refactoring. This change was to organize the Lambdas into their own files, and sort them by the endpoint. Originally, all lambda functions had their code in the same file, making it hard to tell what function did what. Each function was moved into its own file, with a layer being attached to the lambdas allowing for shared code to be used within each function. These Lambdas interact with various resources from the data tier to perform various tasks such as reading and updating DynamoDB, Cognito, and S3. There are also Lambdas that are not attached to the API that get called daily by Event Bridge. The first, Daily Cleaning, serves to clean out old information from the CSV S3 bucket. The second, Simulate Data, runs and simulates hikers for a preset list of trails and puts that data into the database. This allows us to view the application with real information without needing real devices hooked up to it. The data is simulated using an average and std.dev for each trail, with modifications based on how close it is to a holiday and the weather for the day.

The data tier is where all the data that the application manages is stored. Data is primarily stored in four different AWS resources, DynamoDB, S3, Cognito, and Secrets Manager. DynamoDB is where most of the data is stored, with all formatted trail and device data being stored there. Each component on the data model has its own DynamoDB table to store the information for it. S3 is used to store data in the form of files, with an S3 bucket to store CSVs with exported data for the user to data, and CSVs that a user uploads to import data. Cognito stores all the information associated with users, their account id, username, email, password, role, and everything else associated with a specific user. Secrets Manager is used to store all of the certificates used when authenticating with mTLS.

User Interface Design & Architecture



The page transition state model highlights the key pages that users of the web application can interact with. Since the web application was the primary focus of this project, the team decided that this model would be useful in planning how many pages would need to be built, and how well those pages would satisfy the requirements. To briefly describe this model, an actor always enters the application through the landing page, where they have access to the navigation bar. This bar essentially allows navigation to any page, as long as the user has the access level needed to view the page. There are also a few more relationships that don't rely on the navigation bar, such as viewing the dashboard from the landing page, and various popups.

Trail Subsystem

The trail subsystem allows users to add/edit trails from the dashboard. What is referred to as a "trail" is in actuality a trailhead, where the device is placed. This distinction is important as there can only be one trailhead per device, which is a legal constraint and not likely to change, thus the 1-1 associations between Device and Trail. We can guarantee this 1-1 association. Additionally, trail heads have locations that can be added to trails via the add/edit features, and allow the trail to show up on the landing page. These were normally pulled from Google Maps but have recently been sent to us by the sponsor to load into the persistence layer. Trails are associated with Devices which together have usage data, this DeviceTrail usage data can be visualized in multiple ways: via a heatmap view on the landing page, as well as a graph view on the dashboard page. The heat map can be useful for quickly gauging the park or any arbitrary areas usage at a glance, while the dashboard is significantly more helpful for viewing trends and historical data.

Areas Subsystem

Similar to the trail subsystem, areas can be added/edited on the dashboard page by a trail manager and higher privilege. They are groups of trails that have their own location data, though this data is not recorded and is considered a potential enhancement. Areas contain multiple trails and this is used by the dashboard like a filter to view the trails contained in the area on the graph.

User + Privilege Subsystem

The user/privilege subsystem allows users to register and log into the system with their own credentials, and access different features of the application based on their access level. At a baseline, the general public is able to access the application and view pages at a “guest” level. This means they are able to view the lowest access features, such as viewing the map and trails in the Adirondack Pack. Guests are able to either log in or register, and are automatically given “user” level access once they create an account. Currently, the sponsor of the application is given “root admin” privileges, which allows them unrestricted access to all features. They are able to view a privilege management page, where they can view all users who have an account. They are able to promote and demote users, as well as ban and unban users. Users can be promoted to trail managers, and then to admins (not root admin) if the root administrator chooses to. Other admins are also able to view the privilege management page.

Device Management Subsystem

As the number of devices continues to grow, a dedicated device management system is essential for maintaining organization and efficiency. This subsystem was designed based on the sponsor’s wireframes, ensuring the interface aligns with the needs of the primary user. It incorporates all requested data fields, including device name, associated trail, device status, battery level, and last updated timestamp. Additionally, the system provides functionality for adding new devices, supporting scalability as more devices are deployed. Clicking on a specific device shows more information regarding the device, like its id, full device name, notes, and the five most recent device logs. A device can be associated with a trail, archived, or blocked.

Trail Data Subsystem

The trail data subsystem is how all of the information regarding trail counts is stored. It separates the data into four distinct granularities: hourly, daily, weekly, and monthly. Each granularity has a DeviceTrail id representing which DeviceTrail the data is for, a start timestamp representing the start of the given granularity, and a count representing the number of hikers seen during that specific timeframe. A count of zero represents no hikers being spotted during that timeframe, while a missing entry represents a missing upload for that timeframe. The reason for splitting the data into the four different granularities is entirely speed. The application needed to be able to quickly load years worth of data, and we found that by only keeping data in the smallest granularity would take far too long to load. With data retrieved in the highest granularity for the timeframe, the system can remain performant even with a lot of data. This is also the primary factor in the usage of a noSQL DynamoDB instead of a regular SQL database. With little joins

between tables, and all data being queried with exact keys, the timestamp and DeviceTrail id, noSQL was both cheaper and faster for the application.

Device Data Upload Subsystem

The device data upload has changed form significantly since the original variant created in the v1 project we were provided. The original device sent a data packet to the server every time the device was triggered by a hiker walking past it. This created a significant amount of data entries that would bog down the system on recall, and having to manually calculate granularity features(converting the hiker-by-hiker data data to daily, weekly, or monthly formatting) is rather computationally intensive. As such, we strove to cut down on the overhead of such by changing the way devices uploaded data. The v2 device now uploads one data packet on a daily basis(at midnight), containing the timestamp for the data, the number of hikers that passed by for that day, and any additional device data. Sending a single packet is significantly lighter on the system, and for additional load management, we recommended to the device team that each device uploads at +/- 5 minutes to midnight randomly, to ensure that every device isn't uploading at the exact same instant directly at midnight.

On the server end, we now populate device data packets into multiple tables. We have separate tables for each time granularity that we want to support on the website. When data comes in, we populate it into the correct daily/weekly/monthly/yearly table and update the data entries where need be. This allows the granularity processing to be done when the data comes in, and where it only needs to be done for a small portion of data rather than a significant amount at once. Data retrieval for each granularity now all runs at the same speed, and is rather fast.

Registration Subsystem

The registration subsystem is a system built exclusively for the mTLS(mutual transport layer security) protocol. Being able to verify a device communicating to the server is a device that we *expect to* be communicating with the server(and isn't a spoofed device where someone is attempting to exploit our system) is a critical component of the mTLS system. This will not be gone into depth here, and will be elaborated in the security section.

Project Metrics

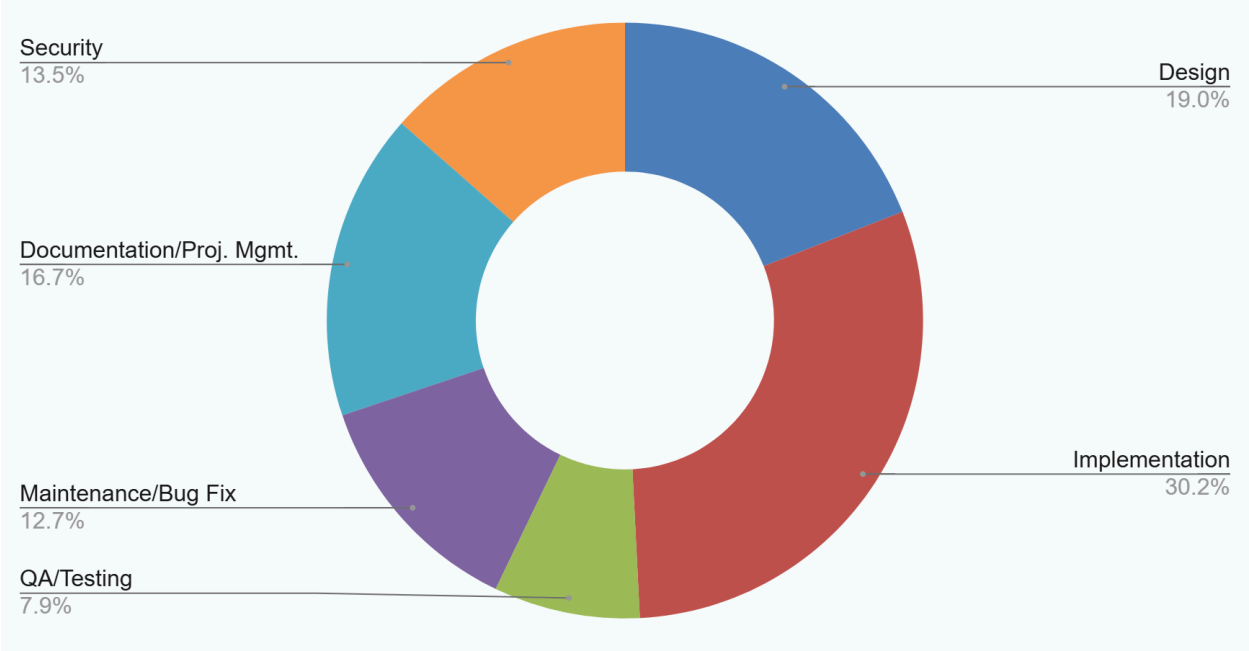
The metrics that the team tracked were time spent working each week, sprint velocity, and types of tasks completed. These metrics were considered to be the most valuable for keeping the team on track. These tasks were consistently gathered over each sprint and over the course of the project.

Regarding time tracking, the team utilized a spreadsheet to track each team member's hours per week. Each team member has their own row to input their own hours, and the idea was to use it similar to a time clock machine. This was used to track individual effort, and to determine if the amount of work designated to each member was balanced or not. There were also other aspects that contributed to these hours, such as some team members not being able to contribute as much on specific weeks due to other responsibilities. As the project progressed, it became clear what

the divide of work should be, and how much work could be reasonably completed by each team member. As a general standard, each member was expected to contribute an average of 10-12 hours per week.

Additionally, sprint velocity was also a metric that the team measured, and it became apparent that finding a sweet spot for velocity would lead to the most success in a sprint. Determining velocity started at the beginning of each sprint, where a dedicated amount of meeting time was set aside to do sprint planning. This involved going through a list of each task, briefly discussing it, and doing planning poker with the entire team. This determined the velocity of each sprint, and this number ranged from 29 to 83, the latter happening towards the end of the project. What the team discovered was that the sweet spot was around the 60-70 range of story points, since that was the greatest number of points that was completed in a sprint without any task slippage. Finding this sweet spot was very helpful for the team to maximize efficiency.

Finally, the last key metric that the team tracked was a task breakdown. Each task on the roadmap was tagged with the type of work it was, and it was graphed on a pie chart to see the types of work that was being completed each sprint.



By inspecting this graph, the team learned that implementation is only a small aspect of the project, and process-related tasks such as documentation and design take up just as much time. While this metric wasn't actively used during the project's development, it is a valuable metric to see, and it provides insight into how important different types of tasks were. The team will absolutely take away lessons after viewing this metric, especially that following a formal process is crucial to a project's success and longevity.

Intern Projects

As the team progressed towards the second half of the project, it was becoming apparent that some requirements were slightly out of reach given the amount of time left, and the amount of

remaining work. Therefore, the team decided to identify two specific areas that two interns could work on. This process involved many steps - the first of which was to identify which aspects of the project could be grouped together. This ultimately decided what skillset we were looking for, and what type of work the interns would be doing (this is detailed below). The second step was indicating our interest in hiring interns, and getting a list of interns to interview. The interview process itself took about 4 hours total, spending about 15-20 minutes interviewing the eight candidates, along with a few minutes of deliberation after each interview. After carefully considering each candidate's skillset and details they were able to provide during the interview, the team selected two candidates who both fit the qualifications and interacted well with the team. The third and final step of the hiring process was writing a work package document, essentially detailing the exact work that they should expect to do.

The interns were expected to quickly get up to speed on the project and its domain, the team's process, and the technologies used to create the application. After joining the team, they have been treated as additional team members, participating in team meetings, sprint planning, demos, and major team discussions and decisions. Each intern was also assigned a coach, who essentially acted as a "supervisor", and were also assigned mentors who were team members they could reach out to and confide in, much like a coworker. Since the team placed such a high emphasis on formal process and procedure, interns would also gain a greater amount of experience working in a software development environment.

Mobile Optimization

One of the interns focused on mobile optimization across the application, which was a highly requested feature by the sponsor. The **Mobile Optimization Intern Work Package**[9] essentially explains that the primary role of this position would be to optimize each webpage for mobile screens, while maintaining the original functionality of the page and not affecting any other views. There were a few deviations from the original work package, mostly due to the lack of written analysis of each page. This was deemed unnecessary because it would introduce more work than needed, and likely reduce the amount of implementation that would have been completed. Additionally, the exact schedule outlined in the work package was not followed, though this was to be expected. Regardless, every aspect of the TRAILS application has been designed and implemented to be mobile-friendly. The process of building a page in mobile started at the design stage, where the page being optimized would be sketched out again, but with a mobile screen size in mind. Then, the coach and possibly another developer would review these designs and provide feedback. The intern would then be able to start the build, creating a branch and implementing the page on their own. From there, it followed the same procedure as the team's code review process.

Device Emulator

The device emulator was a solution we came up with to try and meet the sponsor's definition of "minimum viable product". The sponsor would not accept anything less than demonstrating the full functionality from end to end. With the device team on summer break and resuming in the fall, we have no actual firmware from them, no actual device, and thus no way to provide MVP before we're done. Such would have been absolutely unacceptable, so we needed to create a device emulator to do exactly what the device team is going to do themselves.

The device emulator was intended to serve a few purposes- prove MVP as was already mentioned, test the mTLS code as there was no way to verify it actually worked, and provide a testing framework that we could integrate into more formalized testing to confirm continued functionality. Beyond this, it can also serve as a baseline for the MSD team to base their device firmware off of. If the device emulator works properly, all they need to do is replicate the emulator's output 1:1 in order to build their own firmware, and they can look at the code to see what they need to do.

As such, there were some specific requirements that we had- We needed to be able to simulate everything the device could do, and everything the device *isn't supposed to do*. Simply building out the intended device functionality is useful, but being able to send bad data or certain commands out of order is additionally useful to verify security and that we can handle failures. Being able to test as much as possible is incredibly important.

The implementation of the device emulator includes a Command Line Interface (CLI) which allows the user to create and register devices, upload trail data from a device, manually reregister and renew, using the "demu" command its subcommands. Device creation uses a serialization system to store device information, this device information can then be retrieved by other functions. Device registration is performed via TLS and device uploading is done via mTLS. Options for automatic and manual reregistration and certificate renewal were made with the ability to request a custom expiration date. Additionally there is a command which replicates a connectivity test. All of these features can be built into a Python package.

Commands for the CLI are made by implementing the Command interface and by realizing argument classes. Device serialization is done by automatically writing to and reading from a CSV file. Implementation of TLS and mTLS is done through HTTPS requests formed via the Python Requests module. These requests are then sent to named endpoints on the server appropriate for the task being accomplished. Before any operations that rely on the server can be accomplished, a device has to be registered with the server. When the registration command is performed a private key is generated to sign a CSR (Certificate Signing Request) and then stored locally, the CSR will be touched on in the security section of this document. The certificate the server returns to the program is also stored locally and used to perform mTLS. The data used in uploading and connectivity tests are pulled from JSON files stored locally which are constructed manually. A pyproject.toml file and PDM are used to build the package.

General Assessments

Device Emulator

Device emulator development with the intern went rather smoothly as the intern planned it out remarkably well. The initial architecture diagrams(uml, sequence) and additional planning was incredibly well done by the intern, and provided a strong foundation for actual development to proceed smoothly.

There were some bumps in which device emulator testing/development got caught on because there was an error on the server end, which helped squash a number of bugs present in the backend. This was something we expected to happen, and was one of the reasons we wanted the emulator in the first place.

Some hitches did happen as well since the rest of the team needed to provide certain pieces for the emulator to work, making emulator development dependent on other portions of the project, which increased the time to complete emulator functionality above our initial estimates.

In general, the quality of the emulator code is high and the test suite with it is well written. The emulator was bundled as a python package to make it easier to implement in testing frameworks. We did not manage to get it integrated into our testing pipeline, but strong docs and all the foundations to do such are present, so it should be rather easy for a future developer to do such.

Beyond that, the device emulator is still a useful tool to demonstrate device functionality that can be used to show off and pitch this system to be used on a larger scale. All in all, the software product goes above and beyond our expectations, and the work on it should be commended.

With that said, some of the administrative pieces regarding time management, self motivating to solve problems present in code, and ensuring functionality before deadlines was slightly problematic. The root of these problems does not fall on one individual but rather the full team, where we all were working on various pieces of other work and it resulted in simply expecting the intern to have the work done instead of more frequently checking in and mentoring them to completion, which resulted in the delays here. But, in this regard the intern could have been more proactive on mitigating problems in their work to ensure things went smoother than they did, through getting their work done earlier, reaching out to resolve problems, and ensuring there is sufficient time for the rest of the team to test and approve functionality before demos were due.

Mobile Optimization

As previously mentioned, the mobile optimization work package was well-defined and the process was followed closely. As a result, it led to a successful result, and each page was able to be fully optimized for mobile optimization. Initially, there was a lack of communication between the sub-team (coach and intern) which led to some disconnect and lack of understanding. This was likely due to lack of experience, as the intern and coach were both not used to dynamics like this. This led to some tardiness, especially for the privilege management page, which ended up being pushed back half of a sprint. The mobile sub-team eventually agreed upon weekly mandatory check-ins, as well as being able to reach out to anybody on the team at any point, and this ensured that work was being completed at the expected rate and that there weren't any roadblocks. In terms of wireframes, there was a slight amount of time needed to redesign some of the first wireframes. Without any experience with the team's page design process, this was expected, and accounted for in the work package schedule. Quickly, the intern was able to adapt to the team's style and created designs that fit the standards of the application. One noticeable quality that could have been improved by the intern was speaking up and being a contributing member of the team. This should be taken with a grain of salt, as the intern was online the entire semester, but there was a feeling of a divide between the intern and the rest of the team. This was likely due to many factors: joining the team late, having less experience, being online, and more. But a suggestion for improvement would be to speak up whenever it feels right, and not to be afraid to voice any thoughts.

To summarize, the mobile optimization development went fairly well, and all of the pages that were produced were to the satisfaction of the sponsor. Any problems don't fall on a specific individual, but to the entire team. Again, there is also the factor of being online for the entire semester, while the rest of the team was in-person. This should not be minimized, as working online typically reduces productivity since it increases the amount of distractions and companionship with the team.

Testing

Environments

After reviewing the previous team's work and reflections, one thing that stood out was the lower focus on testing in previous iterations. There was not a lot of testing performed throughout the project's development, and one thing that was prominent in their reflections was that they wished they put more of a focus on testing. This led us to try and put a heavy emphasis on testing throughout the lifecycle of the project.

When the team began to plan the testing infrastructure, we decided that we wanted multiple coexisting versions of the project live at the same time to help test the code at various stages. The main four environments, TST, UAT, PROD, and DEMO, would be live at the same time, and code would be pushed up between them as it gets verified. The first environment, TST, would serve as the developers environment. All development branches would be merged into TST, and when developing the frontend, developers would host the frontend locally while connecting the backend to TST. All changes would be verified in TST before the code is then merged from TST into UAT. The UAT environment exists for the sponsor to review and provide feedback for at the end of every sprint. One of the requests of the sponsor was to be able to view changes to the application and provide feedback on them, so this environment exists to accommodate that request. UAT feedback would then be used to determine tasks for the next sprint. Once the project is ready, the code for UAT is merged into PROD, with DEMO mirroring PROD. PROD exists to be an environment where real devices can connect to and is treated as the functional application rather than a specific testing environment. DEMO exists to mirror the functionality of PROD, but can be tested and used to demo any changes with the device emulator.

To support these environments, the first major code refactor that was performed was to update the project to support having multiple versions of the project hosted on the same AWS account simultaneously. Generally resource names must be unique within an AWS account, so most of the resources were updated to be prefixed with their environment.

Actions

In order to support these environments, and automatically perform tests when moving between them, the team used Github Actions to create a CI/CD pipeline. Each environment is hosted and managed in a terraform cloud, with each environment getting its own workspace. Branch protections were created so that code must flow from TST into UAT into PROD, and all pushes must be in the form of a pull request with two reviews and all actions passing. This pipeline is managed by Github Actions, and a document was created to explain all these actions called the **Github Actions Guide** [4].

The first action is called *Enforce Promotion Pipeline*. This action ensures that all pull requests going into TST are prefixed with "trailblazers", denoting that they are branches that we made.

We did this because there were originally many older branches of the project that we didn't want to immediately remove, so the prefix allowed easier identification of current branches and older ones. The action also ensures that all pull requests going into UAT are from TST, and that all pull requests going into PROD are from UAT.

The second action is called *Terraform Plan*. This action takes the proposed configuration and calls a terraform plan on the terraform cloud workspace. This essentially ensures that terraform would be able to apply the changes if they were to be merged, and that no changes were made to break the terraform. If that succeeds, it then ensures that the frontend can successfully build, ensuring that no changes were made to break the frontend. At the end of the action, it leaves a comment on the pull request that denotes either success or the reason for a failure.

The third action is called *Integration Testing*. This action uses another environment, prefixed with testX, with X being 0-9. The pull request would check out one of the test environments and use it until the pull request is closed. After checking out it builds an environment with the configuration from the pull request, and performs our test suite against the environment, including lambda tests, api tests, and ui tests. The output of these tests is left as a comment alongside a link to view the environment. This link can be used to preview the UI changes on multiple devices without having to download or host the project locally. Once the pull request is closed, it destroys the environment and then releases the workspace. This action, alongside the plan action, handled the continuous integration aspect of the CI/CD pipeline.

The continuous deployment aspect of the CI/CD pipeline was handled by an action called *Terraform Apply*. This action would run any time changes were made to the branches for the TST, UAT, or PROD environments. It would call a terraform apply for the given workspace, and changes applied to that branch would be automatically applied to the correct environment. After successfully applying the changes, the react configuration is then automatically built and deployed to keep the frontend up to date. This allowed changes to be more easily deployed, and environments always stay up to date.

All of the mock data used for testing is stored in various csv files, and can be loaded via a python script. There are ten default trails and devices, and more can be added separately when needed for any specific tests. The default data is approximately three months of simulated trail data for each trail. By storing the data in CSV's, tests are able to read those CSV's and know exactly what data should be presented during the test.

Our test suite is composed of three main types of tests, UI tests, API tests, and Lambda tests. All information about our tests has been documented in the **Testing Guidebook** [5]. The UI tests test the entire system from front to back. There was a set of UI tests inherited with the project, but they have since been completely reworked to a different testing structure. The API tests test the system from the API, with most API tests being inherited and remaining largely unchanged. The Lambda tests test the business logic for all of the lambdas, and are a new addition to the project.

UI Testing

The UI tests for the project follow a conventional UI testing structure, using page objects, step classes, dtos, and the test classes themselves. The goal of the UI tests is to ensure everything flows correctly, and that everything that is displayed to the user matches what should be

displayed in the backend. Breaking down the UI tests into these components takes longer to set up initially, but significantly reduces the time to create new tests and update existing ones to match changed infrastructure.

The first UI testing component is a Data Test Object (DTO). These are objects that serve as a collection of variables that make it easy to pass information that is commonly needed together, and easily test that information. Often one DTO is used as an input to the UI, and another DTO is retrieved from the UI, which are then compared in a test.

The second UI testing component is a Page Object. This is an object that represents a given page on the site. They provide the methods to interact with a given page, that are then used by any given step. A page object first defines all of its html elements at the top of the class, and references them via XPaths. It then has methods to interact with each of the elements in whatever way they need to be interacted with, the being clicked, input to, etc. Page objects are the only places that Selenium is used, interacting with it via a helper class called SeleniumHelper. Selenium is a tool used to control a browser and interact with it in various ways. This helper class uses selenium to perform any actions that a page wants to take, majorly reducing code repetition.

The third UI testing component is a Step Class. This is a class that performs a specific action on one or more page objects. These actions range from very simple, to somewhat complex, and are called by a test in a specific sequence to perform a test. These step classes generally have parameters to determine specific functionality, and attributes to retrieve the results of the step.

The final UI testing component is the Test Class itself. This is where all the testing itself is performed. It calls various steps in a specific order to perform a test. There are two main things that get tested in a given test, that being *the flow* and *the fields*. The flow is ensuring that that process that is expected is what is happening, things like ensuring the test is on the correct page and navigation works properly. This is tested by whether or not selenium throws an error, where if something is supposed to be interacted with and cannot be selenium throws an error and the test saves a screenshot to help debug the issue. The fields are what things are displayed to the user, and verifying that this information is the expected information. This is tested by comparing DTOs retrieved from the frontend with DTOs created from the CSV data.

Lambda Testing

Most of the business logic for the project is within the lambdas that get run. The code for all lambdas for the project is written in python, and all lambdas have their own set of tests (written in pytest) that can be run. These tests are used to ensure that each lambda works correctly in a vacuum, without reliance on anything else. Unlike all other tests in the project, the lambda tests are set up so that they don't require a working application to be run. They can be run standalone meaning almost all of the business logic for the project can be quickly tested with no prior setup.

These tests are commonly considered "unit tests", and this means that these tests were black-box tests that followed the Arrange-Act-Assert (AAA) testing structure. Boundary value analysis was also an emphasized idea, meaning that each lambda test ran through each possible test path and null, invalid, and empty values were all tested. Because of this, there wasn't a strict requirement for code coverage, as long as all core functionality relating to the lambda was tested. All tests

were reviewed by at least three separate people, so nothing important would be missed. Even without a strict coverage requirement, the generated coverage report showed that across all tests there was a coverage of 90%.

Most of the lambdas interact with various data storage services using boto3. In order to perform the lambda testing without relying on these dependencies, the tests all use moto to mock all the services that the lambdas use. All services are mocked before each test case is performed, and filled with data to match the working application. These mock services are then used in place of real ones to create tests with no external dependencies.

Usability Testing

Usability testing was planned and potential users recruited, however we ran out of time to perform the test and had to cut it. We recommend you perform a usability test to determine at the very beginning of the project what UI elements and workflows could be improved.

Manual Acceptance Testing

The team has created a **file**[2] to test the usability of the web application. They are separated by user roles like, guest, trail manager, and admin. This document is meant to provide very little guidance on how to use the application. We will then determine what should be improved based on the results of the testing.

Security

Mutual Transport Layer Security

A significant portion of the project revolved around developing security practices to ensure our system was secure from end to end. The original software system we inherited from the previous team completely neglected security. The API key was locally stored on the device, and the same key was shared by every single device. The github repository did not use environment variables and the api key was stored in plaintext in the repository. If one device got compromised, the entire system was compromised. If the repository went public, the accounts with access to the repository were compromised, or any PC with a cloned copy of the repository was compromised, the entire system would be compromised. A vulnerability like this was determined to be completely unacceptable, and we sought to remedy it.

Our solution was an implementation of mTLS(mutual transport layer security). mTLS fundamentally means that both endpoints confirm the other's validity through a certificate signed by a known certificate authority. Devices must be given a certificate that allows them to identify and authenticate themselves to the server, and the device must also be able to do the same to the server in return. Amazon web services has support for mTLS protocols, but mandates a few prerequisites we had to implement.

Firstly, our website needed to have a custom domain name. This was something that we had already planned on doing, with access to the trailcount.io domain. All we had to do was properly configure the domain via squarespace to map to our APIs.

The next prerequisite required was to have a truststore configured and uploaded to an s3 bucket. This is something that is rather in depth. A truststore is a file that contains what is known as our

“root” certificate. This is what the server uses to verify the device, where any device certificate can be confirmed to be “signed by” the root certificate, and thus trusted. This requires what’s known as a CA(certificate authority) to generate the device certificates using the root certificate, and pass them to the device.

AWS offers a certificate authority service that we could have used, but the usage rates for it were rather steep(we had it ballparked at $400 a month or so) and it was designed for much shorter-lived certificates with expiration dates of a few hours or days. This wasn’t feasible for our use case, firstly because the cost was simply too steep and with the nature of cellular connection with the devices, and the fact that we would be managing our own limited-size network of devices, meaning that a longer term certificate was viable.

As such, we had to create our own certificate authority. Step-CA was the certificate authority software that was chosen for the project, due to its open source nature, docker image support, and all the functionality we required.

The plan was to set up Step-CA in an EC2(elastic compute cloud) instance, which is a virtual machine running on the AWS cloud. The decision was made to seal the virtual machine off from the wider internet, as the network pieces required to hook it up were rather expensive and it would introduce unnecessary security risk that could easily be mitigated, but was wholly unnecessary. A rather convoluted process is used to get the copy of Step-CA from docker into the EC2 environment, utilizing github actions to execute a script to pull it from docker, and upload it to an ECR(elastic container repository), which is an AWS component built for storing docker images. Our EC2 could then check the ECR to see if the docker image was present, and pull it as soon as it was uploaded. Many synchronization pieces were required to ensure that all the proper components were set up and made available to the EC2 before it could begin the setup.

From here, a bash script runs to set up Step-CA, which generates the certificates and configuration files that we need to sign the device certificates. Once it is running and set up, they are stored in AWS secrets manager, which is a file repository for secure data managed by AWS, which is a perfect fit for the private certificates of the CA, something that should never be shared. Secrets manager was set up to never wipe itself on subsequent destructions of the system, so all the data stored there would persist indefinitely. Thus, if we needed to completely restart everything, all the certificates would still be valid and the CA would work as intended, as long as it could restore its config and certificates from the secrets manager copy.

The CA when set up would listen on a port, and our lambda scripts that needed to request certificates could simply send commands to this port to utilize it. As it was sealed off from the wider internet, a VPC(virtual private cloud) needed to be created, which is our own personal network on AWS between whatever components were required. Public/private ips, port forwarding, etc needed to be set up here to give access to everything. The lambdas, the EC2 instance, and a few more components were present in this VPC so they could communicate with each other.

The EC2 instance had SSM(secure session manager) configured so we could connect to the EC2's terminal remotely in order to check logs, debug it, and test functionality as well. This is a feature that AWS has built in, and most of the moving parts to enable it through our VPC were already provided.

Additionally, in practice, there is a quirk of Step-CA that it generates what is known as an "intermediate certificate" which is used to directly sign certificates it creates. The root certificate signs the intermediate certificate, which signs the device certificates. The device certificates can be traced back and verified against either the root or the intermediate certificate For the device certificates, we can additionally configure an expiration date for them, and a lifetime of 720 hours(30 days) was chosen. As connections could be spotty, this felt like enough time for a device to lose connection, and successfully reestablish itself without problem. If a device is offline for more than 30 days, a serious problem is occurring and most likely mandates a manual check of what the current issue is.

The root and intermediate certificates are placed together in a single truststore file, which is then uploaded to a dedicated s3 bucket. This bucket is given to the mTLS secured API to verify the certificates for any incoming requests against. With this in place, mTLS can be enabled on our system.

Now with the CA fully set up and configured inside the EC2, we needed to design a protocol for registering devices, verifying their identity, and securely delivering them their certificates. It was determined that a device would have a "device name" and a "device serial". The name would be the equivalent of a "username" where it is a public identifier used to uniquely identify the device. The serial is the "password" equivalent and should be securely stored. It is used to ensure that the device is who it says it is. Both of these should be flashed into the device alongside the firmware, and additionally noted down somewhere securely for trail managers or administrators to access as need be. The device has a secure memory section where critical values like these can be safely stored and retrieved.

Firstly, the system needs to be made aware of a device. We created a device management page, where new devices could be added. One would input a device name and serial onto the page, and submit it. Both values would be stored in the secrets manager as well, in a key value pair with the name as the key and the serial as the value. As the main site is already secured through https, this submission is properly secured.

Next, the device needs to present itself to the server for its certificate and initial registration. When the device is powered on for the first time, it should attempt to communicate with a standard TLS endpoint for registration. To verify the server's identity during this TLS, AWS offers a set of certificates that systems(in our case, devices) can use to authenticate AWS systems, and these certificates will not expire in any meaningful timeframe(30 years).

Once the TLS handshake is established, the device will send its registration packet to the server. This consists of {{device name, timestamp, CSR(certificate signing request)}, hash}. The dictionary with the 3 objects is the main data packet.

The device name identifies it in the server. If the device name can't be found(if someone skipped the initial device submission before powering the device online, for example), the server rejects the packet.

The timestamp is the time that the device sends the packet. To generate this, the device should query a time server to fetch the unix timestamp, rather than relying on its own internal clock consistently(battery power may lead to inaccuracies). If the time calculated by the server is more than 2 minutes later than the device timestamp, the packet is dropped. This could mean that the packet was intercepted and fiddled with, or some other problem is going on with old packets finally making it to the server. Regardless of the case, it should be invalid.

The next object is the CSR(certificate signing request). This is what a device uses to request a certificate from the CA. It's something most networking libraries can provide to systems, and a CSR is a standard format that's well acknowledged. We include the relevant device name and other information the CA needs, and it generates a certificate for us. If the name in here doesn't match the device name provided, we drop the packet, as this indicates that it's likely been tampered with.

The above 3 pieces are in a separate object in the packet all together. The other object is the hash, which is a HMAC(hash-based message authentication code) hash computed from the {device name, timestamp, CSR} object, hashed against the device serial number as the key, using sha-512 as the algorithm of choice. The goal of this is to ensure that the packet has not been tampered with, and belongs to a real device.

The server also should have the ability to compute this hash on its own. As the original object to hash is provided in the packet, and the server has the serial from the original submission, it can recompute the hash itself. As there is no way to properly modify the hash without access to the original device serial, verifying that the hash is correct is a failsafe way to ensure that the device is who it says it is. The serial is never transmitted between server and device, and any tampering of the data in the packet will mean it will not match the computed hash, causing the packet to be dropped.

Once the server runs all the checks on the packet and verifies to the best of its ability that the device is legitimate, the server sends a request to the CA for a certificate through the port it has access to, providing the CSR. The CA will generate a certificate, and return it. The server will pass that certificate down to the device, and the device will store it for later use. Additionally when it is successful, the CSR the device passed to the server is stored in its secrets manager entry keyed to the device name, alongside the serial.

Once the device has the certificate, it can use it to gain access to the 2 main device API functions: Uploading trail data and renewing its certificate. Both of these are present on a separate API from the registration functionality, as they are secured through mTLS. The device must present its certificate as part of the request. AWS verifies it against the root and intermediate certificates in the truststore, and if they are valid as well as not being expired, the device's request is let through to access the relevant lambda.

Trail data upload works simply as it should, but more elaboration is required on renewal. AWS' recommended time to renew a certificate is 2/3rds through the certificate's lifespan, or in our case, 20 days after the certificate was issued, with 10 full days before it expires. When a device needs to renew, it sends its device name to the renewal endpoint. A device can check the time to live listed on its certificate at any time, and query a time server to determine if it is good to go, needs to renew, or is expired.

The renewal endpoint validates that the name it was sent from the device matches the name on the certificate it was just presented for the mTLS authentication, verifies that it has previously registered, and verifies that the device name exists in the first place. The fact that the device gets through the mTLS auth is nearly enough to verify the device's validity on its own. Note that this will not work for expired or invalid certificates as they cannot get through the mTLS in the first place.

If all the conditions are met, the device's CSR is retrieved from the secrets manager(reusing an old CSR does not cause problems), and a new certificate is requested and issued from the CA. The device will then replace their old one with the new one.

If a device's battery dies or it loses connectivity for the full 10 day renewal window and the certificate expires, it will have to re-register itself. To do such, it simply performs the first time registration once again. This being performed more than once does not cause problems with the system.

This full process ensures that the devices and their workflow is completely secure from end to end for all their operations. The goal for security is to ensure that the system itself is not the weakest link. We have ensured that the device communication with the server is airtight with no possibility to have either side be spoofed by a malicious actor.

With these changes, it is additionally going to be an additional load for the device design team to handle. To accommodate them, we have created a device emulator which was used to simulate device functionality and test the server's security protocols. This will provide the device team with a working "pseudo-device", where all they must do is identically recreate the output and functionality of the emulator.

There is one additional exception to the entire process above. One of our requirements was to provide an adapter for the original "V1" device to work with the system designed for the "V2" devices. As this device is already built, there are some modifications to the current process that need to be made to support it. The data upload was handled in a different format, so a new endpoint on the device api needed to be added. Additionally, the network adapter used on the device does support mTLS, but does not support the root+intermediate certificate pair that we have, and only supports a single certificate. As such, a custom certificate signed directly by the root(without the intermediate like normal devices) needs to be manually generated via SSM-ing into the certificate authority and running some commands to generate it. As this old device cannot be renewed as well, the certificate needs to have a lengthy expiration date so it doesn't need to be updated. As long as this certificate is stored securely, this does not really introduce a

new security risk. The infrastructure to perform and support this has been set implemented, but this procedure has not been performed.

Effort Coordination

A key component of the project's security and overall success was maintaining close coordination with the device (hardware) team to ensure compatibility between their system and our software platform. Because both teams were working on interconnected components, alignment on data formats, communication protocols, and system behavior was essential.

As our team introduced changes to better support security features and evolving requirements, these updates required corresponding adjustments on the device side. While there was some initial hesitation due to the additional effort required, both teams worked collaboratively to evaluate the impact and determine feasible solutions. With guidance from the project sponsor, a shared direction was established that balanced long-term system reliability.

This coordination required ongoing communication, including discussions around API expectations, data structure updates, and deployment considerations. It also influenced how we approached the design of the Device Emulator, ensuring that it remains a useful tool not only for our team but also for the device team as they continue development in future semesters.

Overall, this effort reinforced the importance of cross-team collaboration in building a secure and maintainable system, particularly when multiple components must evolve together.

V1.5 Device Patch

At the time of inheriting the project, a single device had been produced and was deployed. The sponsor made the decision to keep it deployed, so we decided to create a **patch**[8] for this device to bring it into contact with our platform with minimal work. This was much easier than anticipated as it only requires loading the device with long TTL certificates and reloading the modem to use those certificates. Currently the sponsor has not executed the script but it does exist and is possible.

Product State at Time of Delivery

At the time of delivery, the project has been confirmed to completely achieve the MVP, as well as a few enhancements. Features that were proposed enhancements but not added were activity reports, a usability test, and more manual acceptance testing. Other than that, everything that we wanted to accomplish has been completed. This should allow hikers to understand the historical usage of any given trail in the Adirondack Park, give trail maintainers a data-driven platform for seeing which trails need maintenance the most, and policy makers the data they need to make effective policies that'll affect the Adirondack Park.

Future Feature Ideas

Many different potential additional features were either discussed with the sponsor or internally among team members. These features ranged from smaller, "quality of life" additions that were conceptualized from the team's own usage of the website, which notably included the addition of filters to the device table, to more ambitious features that could have been implemented, but were ultimately dropped due to time constraints.

One of the larger planned enhancements was the integration of the landing page and dashboard into a more unified interface. Rather than requiring the user to navigate back and forth between the two pages, the intention was to have the map allow the map displayed on the landing page and the analytical charts on the dashboard to be viewed and interacted with simultaneously. Selecting a trail on the map would automatically update the dashboard's visualizations, and likewise, changing filters, date range, or granularity would immediately update the heatmap on the landing page.

Another larger feature intended for the landing page was to add more comprehensive GIS functionality. This would include having set boundaries for the entire park, as well as displaying defined areas within the park as selectable boundaries. Each area would be labeled and contain the trail pins associated with the trail within its boundaries. When a user selects an area, the map would highlight the boundary, zoom in on the area to the selected area, and display the trails and pins contained within. Trail Managers and admins would also be able to create, edit, and remove these boundaries as needed.

A Trail Report feature was also discussed as a potential approach to condensing the work it might take to interpret dashboard data. The plan is to eventually get to a point where our website is populated with hundreds of data points, so rather than requiring users to analyze all of that raw data themselves, an automated report could be made to interpret larger data statistics into summaries that answer specific questions. These reports could either be made directly through the website's dashboard or be sent to a list of configured users each morning.

Project Reflection

Team Reflection

Some of the most agreed upon successes of our team was its work ethic and our commitment to a workflow that never wavered throughout the entire project's lifespan. Following a formal process was one of the greatest achievements of the team, and contributed to the overall productivity. This would not have been as effective as it was, without every member of the team buying into the process, and each member being internally motivated enough to work as hard as they did. A strong process can only take a group so far, at some point, the individual effort matters just as much if not more.

Another positive reflection is how the team was able to quickly adapt to various changes throughout the project's lifespan. Various challenges were thrown at the team, such as managing other commitments, exams, illnesses, scheduling conflicts, and additional challenges from the sponsor. A way this could be improved further would be to have a way to adapt to problems before they become bigger issues. It would be helpful to have a more formal way to raise potential problems and facilitate discussion about them before they become too big, rather than waiting until the problem forces change. This would help fix issues like the strictly defined developer roles or longer than necessary meetings, before they become big issues.

Personal Reflections

Nico Cabrera - As a whole, this has been an exceptionally valuable and positive experience. As the project manager, I was responsible for making sure that my team got the time they needed, and the leader they deserved. During my time in the military, I learned many right and wrong ways to lead a team, and that showed during the early weeks. I focused too much on the structure and ideal team make up, and less on what people actually wanted to do. It got me in trouble, and I am very glad my team told me about this issue. It helped me a lot to "unclench" so to speak, and trust my team to take on the tasks they needed. My team taught me just as much if not more than the military about managing people. They are individually fantastic engineers, and I'm very happy to have gotten the chance to work with them. In the same vein, I think the military gave me a few valuable things that allowed me to find my place within the team: the knowledge of how to create a structure that allows very intelligent people to do their best work, and the strength and confidence to push back against authority figures. As much as I love the sponsor, there were times when they understandably prioritized business needs over engineering quality and discipline. It was my job as the PM to enforce the team's needs, its need for quality and discipline, even when the sponsor didn't like it. I think the sponsor would agree that despite our initial arguments, the product we produced together has benefited from that. I think that there are certainly areas where I could improve and certainly where my management of the team could improve. Again, I was often far too focused on the optimal solution, and discounted the individual team member. I had a tendency to be draconian, and often took far too long in our meetings. I once promised pizza and did not deliver, I hope my team will forgive me. I think there are plenty of areas for growth that my team have covered below, and I really look forward to improving, and seeing where my team goes from here. I believe that I have made 8 lifelong friends in this class, and I'm very lucky to have been a part of this team.

Fabi Marrufo - Overall, this project was fairly similar to the team projects I have experienced in my classes. However, working with an actual sponsor and another team developing the hardware introduced some challenges that I had not experienced to the same extent in class projects. One of the biggest challenges was coordinating with the hardware team. Since our software needed to work with their device, changes made by one team could affect the other. Communication between the teams was sometimes difficult, and coordinating changes became more of a hassle than expected.

Another challenge was the amount of time spent in meetings. At times, meetings were much longer than necessary, which took away from the time we could spend working on the project.

Working with the sponsor was also different from working with a typical class project. The sponsor was actively involved in developing and shaping the project, and there were times when he was not satisfied with what we had produced.

Richard - I think it would have been beneficial to have a member dedicated to testing, sort of like a Quality Engineer position. Having this person designated to all sorts of testing would have allowed the rest of the team to continue with implementation, and that person would have been able to perform unit, UI, integration, usability, and non-functional testing. I believe this would have helped identify more bugs and usability issues as well, as a tester who does have a software engineering background, not just the sponsor and the other developers occasionally testing pages.

Additionally, I believe that certain parts of our process could have been greatly improved. Across this entire document, there is a strong emphasis on the team's process, which was definitely strong, but I don't think it truly paints the whole picture. For example, having meetings going

more than double its planned length, and lots of inefficiency with sprint planning. There were also many instances of process-related documents that were created, but ultimately weren't used at all, which is essentially wasted time. I think that these unsuccessful documents can be taken as a learning experience though, and it shows why the software development lifecycle is a process shared across the world.

Finally, I wish that I was more vocal during some discussions. There were a few instances where discussions were being had with the sponsor, and looking back on it makes me believe that my input would have helped reach a conclusion sooner. Sometimes I did speak up, but as a general reflection to anybody, I would recommend voicing your opinions if you think they are valuable to the team, because most of the time they will be in some way.

Trevor Wiesen - One thing that I think would've been very helpful in the project was having a member of the team dedicated to managing pull requests, in addition to their other duties. This would mean reviewing every pull request, assigning one or two other members to review it, and setting a date that it needs to be reviewed by. Having someone who is mostly familiar with each aspect of the project to review every pull request would help keep the entire project on the same page. More importantly, it would mean that reviewing a pull request would be done more thoroughly, and avoid having code that is reviewed with a "Looks good to me!" since the deadline for its merge has arrived.

Ethan Patterson - Coming into senior project, the one thing I did not want to do was web development. I'm capable of doing such, but I very much do not care for it and I find it remarkably boring(no offense to the rest of the team that did such). I put that down on my senior project survey form we were made to fill out before teams were chosen, and this was the team I got put on. Everything was just web development. The existence of the "devices" was as close as we could get for non web development, and none of the other summer senior project teams were better. Compared to some of the fall-spring senior project teams where they were designing and developing vr or drone based applications among others, with much more hardware focus and lower level work. I feel like the set of projects for summer was remarkably bland in a way that was a detriment to making the most out of the skillsets of multiple team members on our project, as I'm not the only one on the team with a preference towards the lower level side of software engineering.

I did manage to find my niche with security, as I do enjoy dipping my toes into the cybersecurity side of things(I'm still a software engineer at my core despite this, not a csec major). Being able to fully learn, plan, design, and implement the security protocols on our project from scratch was something I greatly enjoyed. I feel like I learned a lot and it gave me an excellent breadth of experience working firsthand with security in a way I feel would be useful to potentially getting a future career in such a field. I was really left to just "figure it out" on my own as it was rather separated from the rest of the project, and there were multiple sprints where I was mostly just "doing my thing" with little to no oversight from the rest of the team beyond making sure I had a schedule. This was fine as I was able to make it work, but really being the only one who moderately understood what was happening(and even that came at the cost of weeks of google searching and digging through stackexchange posts to try and figure out how to design the system), and then how to even make it all work as it required a lot of convoluted AWS architecture pieces(this project is my first time working with AWS, I had to learn as I went). I

feel like having a larger focus on security(the secure software class covers some things from the code end of things, but is rather shallow on broader security concepts or more specific security techniques imo) in the bachelors curriculum would help. I took csec-202(reverse engineering fundamentals) a few semesters ago and I feel like that course has helped me conceptually understand and design security for this project more than anything I learned from this course.

Additionally, I also had to coach one of the interns, specifically the device emulator intern. While I didn't mind working with the intern, I have a slight problem with the fact that we needed a device emulator at all. The real device is supposed to be developed by a MSD team out of Gleason as their senior project, but they are working spring-fall(wasn't an option for us by the way) and we're working spring-summer. For one thing I very much would have rather worked spring-fall, but at the BARE MINIMUM both teams should have had the same semesters on. The MSD team was already really poor at communicating with us from the start(infrequent messages, some meetings but they felt relatively unproductive, we always had to reach out to them rather than the other way around) to a degree where it felt like we couldn't rely on them. Now with having to finish our project in the summer, which includes changes that they need to abide by to make their project actually work, we have no way to directly guide them through what needs to be done or even realistically know what they can do or can't do. When they come back in the fall, we're *gone*. None of us have any responsibility to help them at that point, and could just leave them to figure it out from our code. Obviously, we're not doing that. We're going to provide them with a full document on everything they need to do with as much hand holding as we can possibly provide. In addition to this, I'm in the combined bs/ms program with another 3 semesters to go, so I will be on campus and I offered to give them an overview in person and be a last ditch break-glass-in-case-of-emergency contact for needing help on what they need. With that said I'm not being paid nor given credit for such, so I'd rather not spoon feed them everything. RIT and the MSD team are incredibly lucky that I'll be around, or else I guarantee that they would be having a horrible time trying to figure things out blindly despite having our documentation.

On a different topic, I made some rather stupid decisions when it came to managing my time with the first semester of the project. Were they something I shouldn't have done? Probably. Would I make the same choices again given the chance? Absolutely. Did I absolutely pay the price for trying to do too much at once? Yes, but I got through it regardless. The senior project timeslot ended up being at the exact same timeslot as orchestra(Mon & Wed 5:00pm-6:15pm) and there were no other time options. On the side of RIT scheduling, this was a very poorly thought out decision to make all the time slots at the exact same date and time so that anyone with conflicts could not manage them. As such, I tried to juggle doing orchestra on mondays, and senior project on wednesdays, and skipping every other class for each one. This *worked* in that I managed to do both at once(neither my orchestra director nor the senior project director was happy about it), and I managed to be passable in both, but barebones passable is not the level I'm comfortable achieving. I had a noticeable decline in my quality of music, and in my quality of my code. Furthermore, this led to some confusion between me and the rest of the team on when I was expected to get things done and how to do my best without impacting the rest of the team. To mitigate this I had to put in extra work for everything, that on top of an already loaded schedule consisting of 5 different musical ensembles, 3 masters classes, senior project, and the courses for a minor, got me dangerously close to burnout. Despite all that, I managed to finish

everything at a quality level that satisfied me, but I had to sacrifice all of my free time for weeks on end in order to make it happen. I worked more on weekends than I did during the week. All of this is mostly the fault of my own actions, and despite some rough starts and misfires with learning how to balance everything, I managed to finish the semester strongly. Once again, I would rather put myself through by far the worst semester of my college career to balance everything rather than drop something I'm genuinely passionate about.

Come summer where I had much less to focus on, I was able to power through much larger and complex workloads at a much faster pace. The summer was fantastic as I only needed to focus on senior project and one other class(analysis of algorithms, probably the worst class I could have taken with this workload wise, but summer still makes it easy). I don't think I would have managed to get the security protocols implemented to the quality that they are right now without the extra time that the summer provided to get our work done.

Giovanni Paulino - Earlier on in the project, I didn't reach out when I needed help with something. I attempted to solve problems I was having by myself. This caused me to fall behind on the implementation of the privileges page and needing to meet with teammates a lot in a short time frame to catch up. Throughout the rest of the project, if I couldn't solve a problem by myself after a few hours, I reached out to a teammate to get their perspective and expertise. This helped the rest of webpage updates go smoothly and be completed on schedule.

In the future, I'll keep in mind that asking for help from a teammate will ultimately be faster for a project than being stuck on a problem on my own.

Conlan Connor - There were times when I wish I had taken more initiative, especially earlier on in the project, to contribute my own ideas into the design of my work. I usually had just trusted the process we had set up to the point where I would never really work beyond what was required of me. I think if I had maybe tried putting in the effort into working on some of my own ideas, even if they weren't ideas previously discussed by the team, we could have had more features done in the frontend.

Lucas French - I think consistently, the fastest way my issues would have been solved is if I had reached out as soon as I had them. The second half of my work on the device emulator heavily involved the program interacting with the server which was a system I hadn't worked on, the nature of this means I was unfamiliar with the architecture being interfaced with and having issues that were not caused by my own code. This is a point where rapid communication would have been incredibly valuable.

References

1. Requirements, Flow, and Use Cases Document (https://docs.google.com/document/d/1c5Fpu7eJF_3tym5sTo9AME3hRtM5MpcTmnyKgls1ymo/edit?tab=t.0)
2. Usability Testing Document (https://docs.google.com/document/d/1_SQvLI6tzEuP3e9CDf-5SgNAyaRTHi6Uu4S8z_7vfFc/edit?usp=sharing)
3. Scope & Requirement Change Request Template (https://docs.google.com/document/d/1vc4f06y2PF8FNnDi8iMKu3n9EyxQUcWbXtMeXhwZVnE/edit?tab=t.0#heading=h.gufrjhw04y0w)

4. Github Actions Guide (https://docs.google.com/document/d/15PRDnog5qQoHh7a94A1kMUg3EhY5dGDc4o9DPUUMORI/edit?tab=t.0#heading=h.2xjwtoihdujw)
5. Testing Guidebook (https://docs.google.com/document/d/1MuMDhpsaSYin0_dwiCYS0pto_3ukT58MzCKLO7v9ang/edit?tab=t.0#heading=h.pgfkkgncll0w)
6. UI Milestones and Work Breakdown (https://docs.google.com/document/d/11gBkHzfovGGH4v7dxtiAKMXFSXC8uxkDcBnuWiOSacI/edit?tab=t.0#heading=h.tmr61g9hn7nh)
7. Github Projects Roadmap (https://github.com/orgs/Trailblazers561/projects/2/views/2)
8. Device Security Workflow Document (https://docs.google.com/document/d/1DpnQ-4-arJpWzvQJWdjkSX5rezbbyQ-cyKcd0zDXVWw/edit?tab=t.0#heading=h.9uidsxsknwfw)
9. Mobile Optimization Intern Work Package (https://docs.google.com/document/d/17QuwrAXVdCk5OR5vxStVbqkCZNkrmwr44V_5kLDLiIg/edit?tab=t.0)